

# gemstone-rs Setup Guide

Install, configure, and complete the first GemStone login



Generated from the Markdown sources in `gemstone-rs/docs`.

# Contents

## Setup Guide

What You Need

Which Install Path Should I Use?

Environment Variables

First Login

Development Checkout

# Setup Guide

---

This guide gets you from a Rust project or source checkout to a live GemStone login.

## What You Need

At minimum:

- Rust stable toolchain
- access to a GemStone/S 64 stone
- GemStone client library access through `libgcirpc`
- GemStone username and password

For the full local development experience:

- a `gemstone-rs` checkout
- `cargo`, `rustfmt`, and `clippy`
- Node.js 22 when working on the VS Code extension
- a Marketplace PAT only when publishing the VSIX
- a crates.io API token only when publishing crates

## Which Install Path Should I Use?

| Use case                    | Command                                                                |
|-----------------------------|------------------------------------------------------------------------|
| Rust application dependency | <code>cargo add gemstone-rs</code>                                     |
| CLI tools                   | <code>cargo install gemstone-rs-cli</code>                             |
| Local HTTP explorer         | <code>cargo install gemstone-rs-explorer</code>                        |
| Source checkout examples    | <code>cargo run -p gemstone-rs --example quickstart</code>             |
| VS Code users               | Install <code>unicompute.gemstone-rs-workbench</code> from Marketplace |

## Environment Variables

Most live commands use `Config::from_env()`:

```
gemstone-rs env sample
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE` is canonical. `GS_STONE_NAME` is accepted as an alias when `GS_STONE` is absent. Setting both to the same value keeps CLI, examples, explorer, and VS Code settings aligned.

`gemstone-rs env sample` prints a safe shell export template. It carries over current non-secret values such as `GS_LIB` or `GS_STONE`, comments optional values when they are absent, and prints placeholders for `GS_PASSWORD` and `GS_HOST_PASSWORD` instead of copying secrets.

Use `gemstone-rs env write` to save the same template:

```
gemstone-rs env write
gemstone-rs env write .env.gemstone-rs --force
```

It refuses to overwrite an existing file unless `--force` is passed. Use the file directly without sourcing it:

```
gemstone-rs doctor --env-file .env.gemstone-rs
gemstone-rs doctor --env-file .env.gemstone-rs --live
gemstone-rs eval --env-file .env.gemstone-rs "3 + 4"
gemstone-rs --env-file .env.gemstone-rs browse dictionaries
gemstone-rs --env-file .env.gemstone-rs codegen check gemstone-rs.codegen
gemstone-rs-explorer --env-file .env.gemstone-rs --port 8787
```

Optional variables:

```
export GS_HOST=localhost
export GS_NETLDI=netldi
export GS_GEM_SERVICE=gemnetobject
export GS_HOST_USERNAME=
export GS_HOST_PASSWORD=
export GS_LIB_PATH=/full/path/to/libgcirpc.dylib
```

| Variable                 | Required | Purpose                                                             |
|--------------------------|----------|---------------------------------------------------------------------|
| <code>GS_LIB</code>      | Usually  | GemStone <code>lib/</code> directory for runtime library discovery. |
| <code>GS_LIB_PATH</code> | No       | Full path to a specific <code>libgcirpc</code> file.                |
| <code>GS_STONE</code>    | Yes      | Stone name.                                                         |

| Variable                    | Required | Purpose                                                     |
|-----------------------------|----------|-------------------------------------------------------------|
| <code>GS_STONE_NAME</code>  | No       | Stone-name alias used when <code>GS_STONE</code> is absent. |
| <code>GS_USERNAME</code>    | Yes      | GemStone login username.                                    |
| <code>GS_PASSWORD</code>    | Yes      | GemStone login password.                                    |
| <code>GS_HOST</code>        | No       | Remote host, defaults to local behavior.                    |
| <code>GS_NETLDI</code>      | No       | NetLDI service name.                                        |
| <code>GS_GEM_SERVICE</code> | No       | Gem service name.                                           |

## First Login

With the CLI:

```
cargo install gemstone-rs-cli
gemstone-rs env sample
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --strict
gemstone-rs doctor --env-file .env.gemstone-rs --live
gemstone-rs --env-file .env.gemstone-rs codegen explain --json gemstone-rs.codegen
gemstone-rs doctor --json
gemstone-rs eval "3 + 4"
```

Expected output:

```
7
```

`gemstone-rs doctor` prints the relevant `GS_*` values with secrets masked, loads the configured `libgcirpc`, and reports setup problems without exposing passwords. The GCI section reports the source that selected the library: explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. Add `--live` when the stone should be reachable; it logs in and asserts that `3 + 4` returns `7`. Add `--json` when a script, CI job, or editor integration needs a parseable report. Failed checks include hints for missing credentials, `libgcirpc` path/loading problems, and live stone connectivity. `--strict` is useful for CI because it also fails when `GS_STONE` / `GS_STONE_NAME` or a deterministic GCI library source ( `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE` ) is missing. The GCI section reports whether the selected `libgcirpc` path exists, is a file, is readable, and whether the path appears to reference arm64 or x86\_64.

From a source checkout:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example hello_gemstone
cargo run -p gemstone-rs --example quickstart
```

From a Rust application:

```
use gemstone_rs::{Config, Session, Value};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env())?;
    assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
    Ok(())
}
```

## Development Checkout

```
git clone git@github.com:unicompute/gemstone-rs.git
cd gemstone-rs
make verify
```

`make verify` runs formatting, `cargo check`, clippy, tests, codegen freshness, the VS Code extension syntax/smoke checks, and PDF generation.

Package the VSIX locally:

```
make vscode-package
```

Verify already-published artifacts:

```
scripts/publish_verify.sh 0.2.2
```

This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.